Name: _____ **Surname:** _____ **Date:** _____

Debugging (15 points each)

Instructions

The functions below contain one or more errors. You have to find these errors, explain why they happen, and suggest a fix to them.

Questions

1. This code is supposed to count down from 5 to 1 and print "Launch!" when the countdown ends. But it never stops.

   - Identify the issue and fix it. *(10 points)*
   - Ensure the count stops at 1. *(5 points)*

```python
def countdown(start: int = 5):
    while start >= 0:
        print(start)
        start + 1
    print("Launch!")

countdown()
```

Answer:

The line `start + 1` calculates a new value but does not store it anywhere, so `start` never changes and the condition of the `while` loop is always true. Besides, we want to count **down**, so the variable has to decrease: `start -= 1`. *(+10 points)*

The condition `start >= 0` makes the loop run also for 0, so it would print 0. To stop at 1, the condition must be `start >= 1`. *(+5 points)*

(Optional, fixed code)

```python
def countdown(start: int = 5):
    while start >= 1:
        print(start)
        start -= 1
    print("Launch!")

countdown()
```

2. The following code is intended to create two sets: one containing the original members of a group, and another containing both the original and new members. It then displays the difference between these sets. However, the output is not as expected. Explain the issue and fix the code.

```
group_before = {"Apple", "Banana", "Carrot"}

# Duplicate the original group and add the new members
group_now = group_before
for new_member in ["Eggplant", "Date"]:
    group_now.add(new_member)

# Compare the groups
print(group_now - group_before)
# Expected output: {"Date", "Eggplant"}
# Actual output: set()  <--- Empty set
```

Answer:

The line `group_now = group_before` does not create a copy of the set. It only creates a second name for the same set object, as sets are mutable. Therefore, adding the new members to `group_now` also adds them to `group_before`, and the difference between a set and itself is the empty set. *(+8 points)*

To fix it, we need to create a real copy of the set, using `group_before.copy()` (or `set(group_before)`). *(+7 points)*

(Optional, fixed code)

```
group_before = {"Apple", "Banana", "Carrot"}

# Duplicate the original group and add the new members
group_now = group_before.copy()
for new_member in ["Eggplant", "Date"]:
    group_now.add(new_member)

# Compare the groups
print(group_now - group_before)  # {"Date", "Eggplant"}
```

3. This function is intended to increment the first element of an array by 1, the second by 2, the third by 3, etc. However, it doesn't return the expected result.

   - Ensure the function outputs an array, instead of `None`. *(5 points)*
   - Make the output array be the expected output. *(5 points)*
   - After you run this function, what will be the value of `x`? *(5 points)*

```python
import numpy as np

def increment_array(arr: np.array):
    length = len(arr)
    for idx in range(length):
        arr[idx] = arr[idx] + idx

x = np.array([1, 0, 1])
y = increment_array(x)

print(y)
# Expected Output: [2, 2, 4]
# Actual Output: None

print(x)  # What will be its value now?
```

Answer:

The function modifies the array `arr` but it has no `return` statement, so it returns `None` by default. We need to add `return arr` at the end of the function. *(+5 points)*

The array must be incremented by 1 in the first position, 2 in the second, etc., but `idx` starts at 0. We need to add `idx + 1`: `arr[idx] = arr[idx] + idx + 1`. *(+5 points)*

After running the fixed function, `x` will be `[2, 2, 4]`. NumPy arrays are mutable and a function receives the array itself, not a copy, so the changes made inside the function also change the original `x` (and `y` is the same array as `x`). *(+5 points)*

Comment: If the function started with `arr = arr.copy()`, `x` would remain unchanged, `[1, 0, 1]`.

(Optional, fixed code)

```python
import numpy as np

def increment_array(arr: np.array):
    length = len(arr)
    for idx in range(length):
        arr[idx] = arr[idx] + idx + 1
    return arr

x = np.array([1, 0, 1])
y = increment_array(x)

print(y)  # [2 2 4]
print(x)  # [2 2 4]
```

4. The following code aims to take a subslice of the array "x". See what the user expects to get, and what is the actual output.

   - Explain briefly what is happening. *(5 points)*
   - Update the code to get the expected output. *(10 points)*

```python
import numpy as np

x = np.array(
    [[10, 20, 30],
     [40, 50, 60],
     [70, 80, 90]])

idx_rows = [0, 2]
idx_cols = [1, 2]

print(x[idx_rows, idx_cols])
# Expected output: [[20, 30], [80, 90]]
# Actual output:  [20, 90]
```

Answer:

When we index a NumPy array with two lists, `x[idx_rows, idx_cols]`, NumPy does not combine all rows with all columns. It pairs the lists element by element, so it picks the positions $(0, 1)$ and $(2, 2)$, which are the values 20 and 90. *(+5 points)*

To select the sub-matrix we can use `np.ix_`, which builds the grid of all the combinations of rows and columns: `x[np.ix_(idx_rows, idx_cols)]`. Another valid option is to index in two steps, `x[idx_rows][:, idx_cols]`. *(+10 points)*

Comment: In the statement, the last argument is written `idx_col`, a typo of `idx_cols`.

(Optional, fixed code)

```python
import numpy as np

x = np.array(
    [[10, 20, 30],
     [40, 50, 60],
     [70, 80, 90]])

idx_rows = [0, 2]
idx_cols = [1, 2]

print(x[np.ix_(idx_rows, idx_cols)])
# [[20 30]
#  [80 90]]
```

Pseudo-Code

Instructions

In the following exercises, you will be asked to write Python functions by hand. Please follow these guidelines:

- Any build-in function you need is in the cheat-sheet provided.
- Clearly indicate each level of indentation using a right arrow (⇒).
- Ensure your code works correctly for any possible example.
- If your code has errors, it will be evaluated based on the provided docstring, comments, and explanations.

Full points will be awarded for code that performs as expected under all conditions.

5. Random Walk (20 points)

You are given a list of player names and a fixed number of steps X. Each player begins at position 0 and performs a random walk of X steps. Write a function `random_walks()` that simulates this process. At each step, the player should either move $+1$ or -1, chosen uniformly at random. After completing the walk for each player, return a dictionary where each key is a player's name, and the corresponding value is that player's final position *(15 points)*.
Your function should accept the following arguments:

- `players` (positional): A list or tuple of player names.
- `num_steps` (keyword): The number of steps for each player's random walk (same number for all of them), defaulting to 10.
- `random_seed` (keyword): If provided, set the random seed to ensure reproducible results.

Additionally, if a player's name appears multiple times in the input list, append a "-" to subsequent occurrences to ensure that all dictionary keys are unique. For example, if "Alex" appears twice, the keys should be "Alex" and "Alex−". If "Bob" appears three times, the keys should be "Bob", "Bob−", and "Bob−−". *(5 points)*
See the code below for an example of input and output:

```
players = ["Alex", "Alex", "Bob", "Bob", "Bob", "Chia"]
x = 10

print(random_walks(players, num_steps=x))
# Example output (actual results will vary due to randomness):
# {"Alex": 2, "Alex-": 0, "Bob": -1, "Bob-": -3, "Bob--": 0, "Chia": 4}
```

Answer:

The function receives the list of players as a positional argument, and the other two as keyword arguments with default values: `def random_walks(players, num_steps=10, random_seed=None)`. If a seed was provided, we set it at the beginning of the function with `np.random.seed(random_seed)`. We must check `if random_seed is not None`, because 0 is also a valid seed. *(+5 points)*

We initialize an empty dictionary `results = {}` and loop over the players. For each player we simulate the walk: we generate `num_steps` random steps, each of them either -1 or $+1$, using `np.random.choice([-1, 1], size=num_steps)`. *(+5 points)*

Because the player starts at position 0, the final position is simply the sum of all the steps, `steps.sum()`. We store it in the dictionary using the player's name as key, and after the loop we return the dictionary. *(+5 points)*

To keep the keys unique, before storing the result we check whether the name is already a key of the dictionary. While it is, we append a "-" to the name. This way, the second "Bob" becomes "Bob-" and the third one becomes "Bob–". *(+5 points)*

(Optional, code)

```python
import numpy as np

def random_walks(players, num_steps=10, random_seed=None):
    # Set the seed only if the user gave one
    if random_seed is not None:
        np.random.seed(random_seed)
    results = {}
    for player in players:
        # Random steps of -1 or +1, and final position
        steps = np.random.choice([-1, 1], size=num_steps)
        final_position = int(steps.sum())
        # Make the key unique
        name = player
        while name in results:
            name = name + "-"
        results[name] = final_position
    return results
```

6. Training Alignment at IE University (20 points)

Each department at IE University offers a list of training modules. Some team members cannot attend certain modules because they have already completed them. Your task is to find which modules can be attended by everyone in each department.
Create a function `modules_for_team()`. It takes two inputs:

- `modules_by_dept`: a dictionary where keys are department names and values are lists of module titles.
- `employee_opt_outs`: a list of tuples with employee names and the modules they cannot attend.

The function `modules_for_team()` returns a dictionary with each department's module list filtered so that only universally acceptable modules remain. *(15 points)*
If a department ends up with no options for everyone, it is not included in the output. *(3 points)*
Also, ensure the original inputs remain unchanged after running your function. *(2 points)*
See the code below for an example of input and output:

```
modules_by_dept = {
    "HR": ["Harmony", "Inclusion", "HybridOps"],
    "Tech": ["Coding", "CloudTech", "Scrum"],
    "Admissions": ["CRM", "Recruitment", "Outreach"]
}

employee_opt_outs = [
    ("Sonia", ["Inclusion", "Scrum"]),
    ("Jose", ["Harmony", "HybridOps"]),
    ("Angel", ["CloudTech"]),
    ("Yuriko", ["Recruitment", "CloudTech"]),
    ("Sara", ["Inclusion", "CRM"]),
]

modules_viable = modules_for_team(modules_by_dept, employee_opt_outs)
print(modules_viable)
# Expected output:
# {
# "Tech": ["Coding"],
# "Admissions": ["Outreach"]
# }
```

Answer:

The function takes the dictionary of modules by department and the list of tuples with the opt-outs, and returns a new dictionary. First, we collect all the modules that at least one employee cannot attend. We create an empty set, `blocked = set()`, loop over the tuples with `for name, modules in employee_opt_outs`, ignore the name, and add the modules to the set with `blocked.update(modules)`. A set is a good choice because it avoids duplicates and checking `in` is fast. *(+5 points)*

Then we create an empty dictionary for the output and loop over the departments with `for dept, modules in modules_by_dept.items()`. For each department we build a new list containing only the modules that are not in `blocked`. *(+10 points)*

If this new list is empty, we skip the department. Otherwise, we store the list in the output dictionary using the department name as key. *(+3 points)*

We never modify the input dictionary or its lists: the filtered modules go into new lists, and the opt-outs are only read. Therefore, the original inputs remain unchanged. *(+2 points) (Optional, code)*

```python
def modules_for_team(modules_by_dept: dict, employee_opt_outs: list) -> dict:
    # Modules that at least one employee cannot attend
    blocked = set()
    for name, modules in employee_opt_outs:
        blocked.update(modules)

    # Keep the modules that everybody can attend
    viable = {}
    for dept, modules in modules_by_dept.items():
        ok_modules = []
        for module in modules:
            if module not in blocked:
                ok_modules.append(module)
        # Departments without options are not included
        if len(ok_modules) > 0:
            viable[dept] = ok_modules
    return viable
```

Cheatsheet

Given a number named `x`:

- `x % y` returns the remainder of the division $\frac{x}{y}$.
- `x // y` returns the quotient of the division $\frac{x}{y}$.
- `abs(x)` returns the absolute value of `x`.

Given a string named `s`:

- `int(s)` returns an integer. Fails if `s` has invalid characters.
- `float(s)` returns a floating number. Fails if `s` has invalid characters.
- `s.lower()` returns a copy of the string, all in lowercase.
- `s.split(" ")` split a string into a list where each word is a list item.
- `s.count(<pattern>)` returns the number of non-overlapping occurrences of `<pattern>`.
- `s.find(<pattern>, <start>)` finds the first occurrence of the specified `<pattern>`, starting from index `<start>`. Returns `-1` if the pattern is not found.
- `s.replace(<pattern>, <new>)` returns a new string where all instances of `<pattern>` have been replaced with `<new>`.

Given a list named `ls`:

- `ls.append(element)` add element to that list.

Given a list or tuple named `ls`:

- `ls.index(element)` returns the index of element in that list.
- `len(ls)` returns the number of elements in that list.
- `sum(ls)` returns the sum of the elements of that list.
- `sorted(ls)` returns a sorted version of that list.
- `tuple(ls)` / `set(ls)` returns a tuple / set using the elements of that list.
- `", ".join(ls)` returns a string with the elements of the list separated by commas.

Given a set named `a`:

- `a.add(x)` adds the element `x` to the set `a`.
- `a.union(b)` or (`a | b`) returns the union of set `a` with another set `b`.
- `a.intersection(b)` or (`a & b`) returns the intersection of set `a` with another set `b`.
- `a.difference(b)` or (`a - b`) returns the difference of set `a` with another set `b`.

Given a dictionary named `d`:

- `list(d.keys())` returns a list with the keys of `d`.
- `list(d.values())` returns a list with the values of `d`.
- `list(d.items())` returns a list with the tuples (key, value) of `d`.

Some useful build-in functions:

- `type(x)` returns the type of `x` (int, float, list...).
- `range(start, stop)` generates numbers from `start` until `stop`, not including `stop`.
- `input(msg)` is used to take user input, in form of a string.

NumPy Cheatsheet

Importing NumPy:

- `import numpy as np`
- `rng = np.random.default_rng()`

Creating arrays:

- `np.zeros((m, n))` creates an $m \times n$ array filled with zeros.
- `np.ones((m, n))` creates an $m \times n$ array filled with ones.
- `np.arange(start, stop, step)` creates values from `start` to `stop` (exclusive) with a given `step`.
- `np.linspace(start, stop, num)` creates `num` evenly spaced values between `start` and `stop`.

Given a NumPy array `arr`:

- `arr.ndim` returns the number of array dimensions, as a number.
- `arr.shape` returns the shape of an array, as a tuple.

Array operations:

- `np.dot(a, b)` computes the dot product.
- `np.matmul(a, b)` or `a @ b` performs matrix multiplication.
- `np.transpose(a)` or `a.T` transposes the array.

Array manipulation:

- `a.reshape((m, n))` changes the shape of an array.
- `a.flatten()` flattens a multidimensional array into 1D.
- `np.concatenate((a, b), axis=0)` joins arrays along an axis.

- `np.stack(<sequence>)` joins a sequence of arrays along a new axis.
- `np.vstack((a, b))`, `np.hstack((a, b))` vertical/horizontal stacking.
- `np.split(a, n)` splits an array into `n` parts.

Statistics and descriptive analysis:

- `np.mean(a)` mean of all elements.
- `np.median(a)` median of all elements.
- `np.std(a)` standard deviation.
- `np.var(a)` variance.
- `np.min(a)`, `np.max(a)` minimum and maximum.
- `np.argmin(a)`, `np.argmax(a)` indices of minimum and maximum.
- `np.corrcoef(a, b)` correlation coefficient matrix.
- `np.cov(a, b)` covariance matrix.

Linear algebra:

- `np.linalg.inv(a)` inverse of matrix `a`.
- `np.linalg.det(a)` determinant of `a`.
- `np.linalg.eig(a)` eigenvalues and eigenvectors.
- `np.linalg.norm(a)` vector or matrix norm.
- `np.linalg.solve(a, b)` solves $Ax = b$.

Logical operations and conditions:

- `np.where(cond, x, y)` selects elements from `x` or `y` based on condition.
- `np.all(a)`, `np.any(a)` check if all/any elements are True.
- `np.unique(a)` returns sorted unique elements.

Random number generation (using `rng`):

- `rng.random(size)` uniform distribution in $[0, 1)$.
- `rng.integers(low, high, size)` random integers in $[low, high)$.
- `rng.normal(mean, std, size)` normal (Gaussian) distribution.
- `rng.uniform(low, high, size)` uniform distribution in $[low, high)$.
- `rng.choice(a, size)` random sample from array `a`.

Random number generation (legacy API):

- `np.random.randint(low, high, size)` returns random integers from low (inclusive) to high (exclusive).
- `np.random.uniform(low, high, size)` samples from a uniform distribution.
- `np.random.seed(seed)` fixes the seed. After a seed reset, the same set of numbers will appear every time.

Math and trigonometry:

- `np.power(arr1, arr2)` returns the first array elements raised to powers from second array, element-wise.
- `np.sin(arr)` returns the trigonometric sine, assuming angle in radians.
- `np.cos(arr)` returns the trigonometric cosine, assuming angle in radians.
- `np.radians(arr)` converts angles from degrees to radians.
- `np.degrees(arr)` converts angles from radians to degrees.